



DRISHTI
A Revolutionary Concept

MAKERNOVA 4.0

PS4 Controller or Web GUI Controlled
Omnidirectional Robot

Problem Statement

Develop a 3-wheel omnidirectional robot enabling holonomic motion using inverse kinematics. Implement dual wireless control through a PS4 controller and a Web GUI, with real-time communication to an ESP32 over WebSocket. An IMU sensor is integrated for orientation estimation, and a PID controller is implemented to maintain heading stability, improving navigation accuracy and overall motion control.

Mentors :

- Bhavya Desai
- Hitesh Bihani
- Mehak Oberoi
- Prem Vyas
- Shubham Kumar

Students :

- Ankita Mina
- Ashish Reddy
- Chetan Suthar
- Gladwin Britto
- Jay Rathod
- Jenil Jetani
- Neel Limaye
- Panthak Shah
- Parth Ghaskata
- Rakshit Parikh
- Rohit Gohil
- Vishesh Bindal
- Viral Patel

Acknowledgment

We express our sincere gratitude to SVNIT for providing us with the opportunity and resources to undertake this project as part of Makernova. We are deeply grateful to our project mentors for their valuable guidance, continuous support, and constructive suggestions throughout the development of our 3-Wheel Omnidirectional Robot. Their knowledge and encouragement helped us understand concepts such as inverse kinematics, wireless communication, motor control, and ESP32-based robotics.

Abstract

The 3-Wheel Omnidirectional Robot is a compact mobile robot designed for easy movement in different directions. Its three omni wheels allow it to move forward, backward, sideways, and diagonally, and to rotate without changing the orientation of the chassis. An ESP32 DevKit V1 is used as the main controller, and the robot can be controlled wirelessly using either a PS4 controller or a Web GUI, with WebSocket communication providing fast interaction between the GUI and the ESP32. Inverse kinematics is used to convert the desired robot movement into individual wheel speeds, which are then applied through the motor drivers. The robot can perform basic movements such as forward, backward, left, right, clockwise, and counterclockwise rotation. Overall, the project combines embedded systems, wireless communication, motor control, and inverse kinematics to build a practical omnidirectional robot.

Table of Content

1	Introduction	6
1.1	Project Overview	6
1.2	Types of Motion	6
1.2.1	Holonomic Motion	6
1.2.2	Non-Holonomic Motion	6
1.3	Types of Wheels	7
1.3.1	Omni Wheels	7
1.3.2	Mecanum Wheel	7
1.4	Microcontroller, Microprocessor, and Development Board	8
1.4.1	Microcontroller	8
1.4.2	Microprocessor	8
1.4.3	Development Board	8
2	Theoretical Study and Kinematics	9
2.1	Forward Kinematics	9
2.2	Inverse Kinematics	9
3	Components	11
3.1	Microcontroller ESP32	11
3.2	Motor	11
3.3	Motor Driver	12
3.3.1	Rhino MDD20Amp DC Motor Driver (RMCS2305)	12
3.3.2	MD13S	12
3.4	IMU	12
3.5	Battery	13
3.6	LM2596 Buck Converter	14
4	Circuit Diagram and Control Circuit	15
4.1	Power and Control Circuit	15
4.1.1	Power Circuit	15
4.1.2	Control Circuit	15
4.1.3	Circuit Separation	15
5	Web GUI and WebSocket	16
5.1	Web GUI	16
5.2	WebSocket	16
6	Communication Protocols	17
6.1	What are Communication Protocols?	17
6.2	Serial Peripheral Interface (SPI)	17

6.3	I ² C	17
6.4	UART	18
6.4.1	UART Configuration	18
7	PID	19
7.1	Proportional Controller	19
7.2	Integral Controller (I)	19
7.3	Derivative Controller	20
7.4	Combining P, I and D	20
8	References	22

List of Figures

1	3-Wheel Omnidirectional Bot	6
2	Omni wheel with passive rollers	7
3	Mecanum wheel	7
4	Vector Diagram	9
5	ESP32	11
6	DC Planetary Gear Motor	11
7	Rhino MDD20Amp Motor Driver	12
8	MD13S Motor Driver	12
9	MPU6050	12
10	BNO085	13
11	Battery	13
12	LM2596 Buck Converter	14
13	Circuit Diagram	15
14	GUI interface	16
15	Block Diagram of PID	20
16	Graph of PID	21

List of Tables

1	Comparison Between Omni Wheels and Mecanum Wheels	8
2	Characteristics of Proportional Gain (K_p)	19
3	Characteristics of Integral Gain (K_i)	20
4	Characteristics of Derivative Gain (K_d)	20

1. Introduction

1.1 Project Overview

This project focuses on developing a compact three-wheeled omnidirectional robot capable of moving in different directions without changing the orientation of its body. An ESP32 micro-controller serves as the main controller of the robot.

The project involved both hardware and software development. On the hardware side, the robot uses three omni wheels, motors, motor drivers, and sensors, while the software handles movement, wireless control, and feedback from the sensors. The robot can be controlled wirelessly through either a PS4 controller or a web-based control interface.

Inverse kinematics is used to calculate the required speed and direction of each wheel based on the robot's desired movement, while an IMU measures its orientation. A PID controller reduces unwanted rotation or drift and helps the robot maintain the required heading during movement. Bringing all these components together, we were able to build and test a working three-wheeled omnidirectional robot.

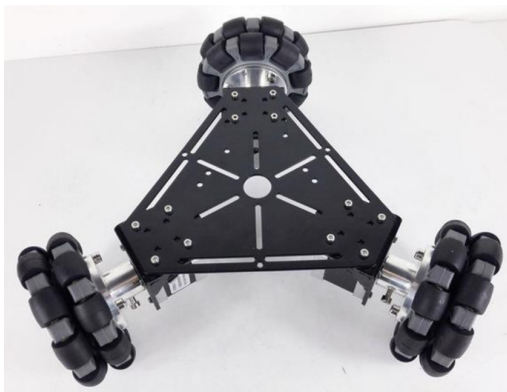


Figure 1. 3-Wheel Omnidirectional Bot

1.2 Types of Motion

1.2.1 Holonomic Motion

Holonomic motion refers to the ability of a robot to independently control all of its available degrees of freedom. For a robot moving on a flat surface, these are generally:

V_y - velocity along the y-axis

V_x - velocity along the x-axis

ω - angular velocity

Our three-wheeled omni robot is holonomic because it can control these three components of motion independently. This allows it to move in any direction while also rotating about its center.

Omni Motion

Omni motion allows a robot to move in different directions without first changing the orientation of its chassis. It is achieved using omnidirectional wheels, which have small rollers around their circumference. An omni-wheel robot can move forward, backward, sideways, and diagonally. It can also translate and rotate at the same time. In our project, the three omni wheels are positioned approximately 120° apart, which allows the robot to achieve multi-directional movement. The ESP32 uses inverse kinematics to calculate the required speed and direction of each motor based on the desired movement.

1.2.2 Non-Holonomic Motion

A non-holonomic robot cannot move freely in every direction, since its wheel arrangement restricts its movement. For example, a differential-drive robot cannot move directly sideways; it must rotate first and then move in the required direction. Differential-drive, Ack-

ermann, and skid-steer robots are common examples of non-holonomic systems.

1. Differential Motion

Differential motion is commonly used in mobile robots with two independently driven wheels. The robot changes direction by varying the speed of the left and right wheels: when both wheels move at the same speed, the robot moves straight, while different speeds produce a turning motion. If the wheels rotate in opposite directions, the robot can rotate about its center. However, a differential-drive robot cannot move directly sideways, so it is considered non-holonomic.

2. Ackermann Motion

Ackermann motion is based on the steering mechanism used in conventional cars. The front wheels are used for steering, while the drive wheels provide forward or backward movement. By changing the steering angle, the robot can follow curved paths. This type of motion is suitable for car-like robots but does not allow direct sideways movement or rotation in place.

3. Skid-Steer Motion

Skid-steer robots use multiple wheels arranged along the left and right sides of the chassis. The robot changes direction by controlling the relative speed of the wheels on each side. For example, if the wheels on one side rotate faster than the other, the robot turns; if the two sides rotate in opposite directions, the robot rotates around its center. During turning, the wheels tend to skid against the ground, which creates additional friction.

1.3 Types of Wheels

1.3.1 Omni Wheels

Omni wheels are conventional wheels fitted with small passive rollers around their circumference, with the rollers mounted perpendicular

to the main wheel's driving direction.

The main wheel provides traction for forward and backward motion, while the rollers allow the wheel to move freely sideways. This enables omnidirectional/holonomic motion, allowing a robot to move forward, sideways, and diagonally, and to rotate without changing its orientation.



Figure 2. Omni wheel with passive rollers

The main advantages of omni wheels are high maneuverability, a small turning radius, precise movement, a compact design, and the ability to move and rotate simultaneously, which make them particularly suitable for an omnidirectional robot.

1.3.2 Mecanum Wheel



Figure 3. Mecanum wheel

A Mecanum wheel also has rollers, but the major difference is that its rollers are mounted

at approximately 45° relative to the wheel axis. Mecanum robots usually use four independently driven wheels.

Mecanum wheels are also holonomic and provide omnidirectional motion, but they generally require a four-wheel arrangement with rollers mounted at an angle, commonly 45° . This adds extra motors, motor-control channels, mechanical components, and synchronization requirements. For this reason, omni wheels were considered more appropriate for the proposed compact omnidirectional robot.

Table 1. Comparison Between Omni Wheels and Mecanum Wheels

Feature	Omni Wheels	Mecanum Wheels
Roller Angle	Rollers are placed at 90° .	Rollers are placed at 45° .
Wheel Setup	Wheels can be arranged in different ways, such as X-drive or H-drive.	Four wheels are normally arranged in a standard X-pattern.
Wheel Type	The same type of wheel can be used throughout.	Left- and right-handed wheels are required.
Minimum Wheels	Can provide omnidirectional motion with 3 wheels.	Usually needs 4 wheels for omnidirectional motion.
Forward Motion	More efficient, since the driving force acts directly.	Slightly less efficient due to the 45° roller angle.
Space Required	Compact and suitable for smaller chassis.	Requires more space because of its wheel arrangement.
Movement Smoothness	Generally provides smooth movement.	Can produce slight vibration while moving.
Best Suited For	Lightweight and highly agile robots.	Heavier robots and industrial applications.

1.4 Microcontroller, Microprocessor, and Development Board

1.4.1 Microcontroller

A microcontroller (MCU) is a small computer on a single integrated circuit designed to con-

trol specific tasks within electronic systems. It combines the functions of a central processing unit (CPU), memory, and input/output interfaces on a single chip.

A typical microcontroller consists of a processor core, volatile and non-volatile memory, input/output peripherals, and various communication interfaces. The processor core is responsible for executing instructions and controlling the other components of the microcontroller.

The memory stores data and program code, while the input/output peripherals allow the microcontroller to interact with the external environment.

1.4.2 Microprocessor

A microprocessor is a programmable device that takes in input, performs arithmetic and logical operations on it, and produces the desired output. In simple terms, a microprocessor is a digital device on a chip that can fetch instructions from memory, decode and execute them, and return the result. It is a fundamental part of a computer's architecture, without which the computer would not be able to perform any operations.

1.4.3 Development Board

A development board is an all-in-one platform containing a microcontroller and essential support hardware, allowing users to prototype and program electronics without building custom circuits from scratch.

Key Benefits:

- **Ready-to-Use Platform:** Includes power regulation, built-in programming interfaces, and accessible pinouts out of the box.
- **Faster Development:** Eliminates breadboard complexity so you can focus immediately on writing code and testing ideas.

2. Theoretical Study and Kinematics

2.1 Forward Kinematics

Forward kinematics (FK) performs the reverse operation of inverse kinematics. It determines the robot's overall global motion (task space) by analyzing the actual, real-time speed of each individual wheel (joint space).

System Inputs

To calculate the robot's trajectory, the FK model takes the current velocities of the individual wheels as inputs.

2.2 Inverse Kinematics

Inverse kinematics (IK) serves as the mathematical bridge that translates the robot's desired global movement (task space) into specific rotational speeds and directions for each individual wheel (joint space).

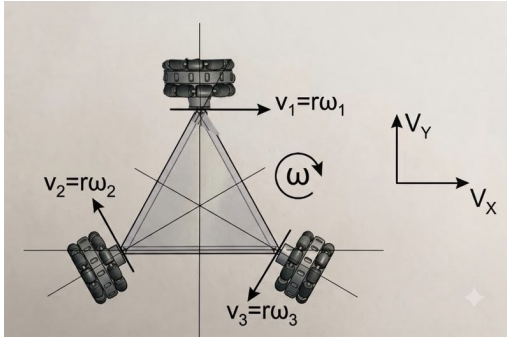


Figure 4. Vector Diagram

For an omnidirectional wheel, the total spinning speed of the wheel ($r\omega_i$) is the sum of the robot's translational velocities (V_x and V_y) projected onto the wheel's driving angle (θ_i), combined with the tangential velocity caused by the robot's rotation (ω).

The general equation for any wheel i is:

$$r\omega_i = V_x \cos(\theta_i) + V_y \sin(\theta_i) + \omega R$$

Where:

V_x is the linear velocity along the positive X-axis (Right).

V_y is the linear velocity along the positive Y-axis (Forward).

ω is the angular velocity of the robot around its center (Clockwise).

R is the distance from the robot's center to the wheel.

r is the radius of the wheel.

θ_i is the angle of the wheel's driving direction relative to the positive X-axis.

1. Derivation for Wheel 1

Wheel 1 is located at the top of the robot and drives parallel to the negative X-axis.

Driving Angle: $\theta_1 = 180^\circ$

We project the robot's target velocities onto Wheel 1:

X-axis contribution:

$$V_x \cos(180^\circ) = V_x(-1) = -V_x$$

Y-axis contribution:

$$V_y \sin(180^\circ) = V_y(0) = 0$$

Rotation contribution: ωR

(Although the wheel is tangent to the rotation, its driving direction directly opposes clockwise motion, so it takes on 100% of the rotational velocity in the negative direction.)

Summing these contributions yields the equation for Wheel 1:

$$r\omega_1 = -V_x + \omega R$$

2. Derivation for Wheel 2

Wheel 2 is located at the bottom-left. Since the three wheels are spaced evenly (120° apart), its driving direction is rotated 120° counterclockwise from Wheel 1.

Driving Angle: $\theta_2 = 300^\circ$

We project the robot's target velocities onto Wheel 2:

X-axis contribution:

$$V_x \cos(300^\circ) = V_x(0.5) = 0.5V_x$$

Y-axis contribution:

$$V_y \sin(300^\circ) = V_y \left(-\frac{\sqrt{3}}{2}\right) = -\frac{\sqrt{3}}{2}V_y$$

Rotation contribution: ωR

Summing these contributions yields the equation for Wheel 2:

$$r\omega_2 = 0.5V_x - \frac{\sqrt{3}}{2}V_y + \omega R$$

3. Derivation for Wheel 3

Wheel 3 is located at the bottom-right, rotated another 120° counterclockwise from Wheel 2 (since $300^\circ + 120^\circ = 420^\circ \equiv 60^\circ$).

Driving Angle: $\theta_3 = 60^\circ$

We project the robot's target velocities onto Wheel 3:

X-axis contribution:

$$V_x \cos(60^\circ) = V_x(0.5) = 0.5V_x$$

Y-axis contribution:

$$V_y \sin(60^\circ) = V_y \left(\frac{\sqrt{3}}{2}\right) = \frac{\sqrt{3}}{2}V_y$$

Rotation contribution: ωR

Summing these contributions yields the equation for Wheel 3:

$$r\omega_3 = 0.5V_x + \frac{\sqrt{3}}{2}V_y + \omega R$$

Final Matrix Formulation: We can now compile the three individual wheel equations into a single system of linear equations. By separating the constants from the velocity variables (V_x , V_y , ω), we construct the final kinematics matrix:

$$\begin{bmatrix} r\omega_1 \\ r\omega_2 \\ r\omega_3 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ 0.5 & -\frac{\sqrt{3}}{2} & 1 \\ 0.5 & \frac{\sqrt{3}}{2} & 1 \end{bmatrix} \begin{bmatrix} V_x \\ V_y \\ \omega R \end{bmatrix}$$

Factoring out the wheel radius (r) to solve directly for the wheel angular velocities

($\omega_1, \omega_2, \omega_3$), we get the standard form:

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} -1 & 0 & R \\ 0.5 & -\frac{\sqrt{3}}{2} & R \\ 0.5 & \frac{\sqrt{3}}{2} & R \end{bmatrix} \begin{bmatrix} V_x \\ V_y \\ \omega \end{bmatrix}$$

Generalized Matrix Formulation

Before plugging in the specific wheel angles, the inverse kinematics can be represented in a generalized matrix format. Using the general equation derived earlier for clockwise rotation:

$$r\omega_i = V_x \cos(\theta_i) + V_y \sin(\theta_i) + \omega R$$

We can stack the equations for any three wheels ($i = 1, 2, 3$) into a single linear system. By keeping the driving angles as variables ($\theta_1, \theta_2, \theta_3$), the generalized kinematics matrix is:

$$\begin{bmatrix} r\omega_1 \\ r\omega_2 \\ r\omega_3 \end{bmatrix} = \begin{bmatrix} \cos(\theta_1) & \sin(\theta_1) & 1 \\ \cos(\theta_2) & \sin(\theta_2) & 1 \\ \cos(\theta_3) & \sin(\theta_3) & 1 \end{bmatrix} \begin{bmatrix} V_x \\ V_y \\ \omega R \end{bmatrix}$$

Factoring out the wheel radius (r) gives the fully generalized form for the wheel velocities:

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} \cos(\theta_1) & \sin(\theta_1) & 1 \\ \cos(\theta_2) & \sin(\theta_2) & 1 \\ \cos(\theta_3) & \sin(\theta_3) & 1 \end{bmatrix} \begin{bmatrix} V_x \\ V_y \\ \omega R \end{bmatrix}$$

The 3-wheel configuration was selected for this project because it provides the required omnidirectional movement while keeping the robot simple, lightweight, and relatively easy to control. Using three wheels also reduces the number of motors and hardware components required compared to a 4-wheel design.

3. Components

3.1 Microcontroller ESP32

The ESP32 is a low-power, low-cost, energy-efficient microcontroller. It was designed by Espressif Systems and is manufactured by TSMC using their 40 nm process. It can be programmed to communicate with sensors and is a system-on-chip (SoC) that integrates multiple hardware peripherals onto a single integrated circuit, enabling embedded real-time control and wireless communication.

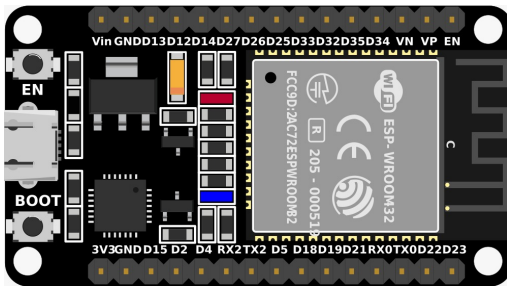


Figure 5. ESP32

Salient Features:

- Processor: 32-bit dual-core processor running up to 240 MHz.
- Connectivity: Built-in 2.4 GHz Wi-Fi and Bluetooth module.
- Memory: Integrated SRAM with support for external flash memory.
- Peripherals and I/O: Multiple GPIO pins featuring ADC (Analog-to-Digital) and DAC (Digital-to-Analog) capabilities.
- Communication: Supports UART, SPI, and I²C protocols.
- Hardware interrupt support for fast response to external events.
- Features hardware encryption, secure boot, and a unique 48-bit MAC address.
- Suitable for web servers, WebSockets, sensor interfacing, and compact embedded electronic designs.

Boards and Libraries Used in Arduino IDE

- **Boards:**
 - esp32 by Espressif Systems (Version: 2.0.7)
- **Libraries:**
 - Adafruit BNO08x by Adafruit
 - Adafruit MPU6050 by Adafruit
 - PS4Controller by Juan Pablo Marquez
 - WebSockets by Markus Sattler

3.2 Motor

A motor acts as the "muscle" of a robot, converting electrical power into physical movement. This specific motor is designed for heavy-duty automation and high-torque applications.

Key Specifications:

- Operating Voltage: 18 VDC
- Power Output: 250 W
- Speed: 260 RPM (Rated) | 300 RPM (No-load)
- Torque: 92 kg·cm (Rated) | 200 kg·cm (Maximum breaking torque)
- Current Draw: 2.7 A (No-load) | 18.7 A (Rated) | 130 A (Stall/Peak)
- Dimensions: 46 mm diameter × 125 mm body length
- Hardware: 8 mm hardened shaft with an M4 mounting pattern



Figure 6. DC Planetary Gear Motor

3.3 Motor Driver

3.3.1 Rhino MDD20Amp DC Motor Driver (RMCS2305)

Rhino Dual Driver 20A Specifications:

- Core Function: Bidirectional control for two high-power brushed DC motors.
- Operating Voltage: 6V to 30V.
- Current Capacity: 20A continuous per motor (no heat-sink required); 60A peak (for up to 10 seconds).
- Control Interface: Accepts standard PWM and DIR inputs.

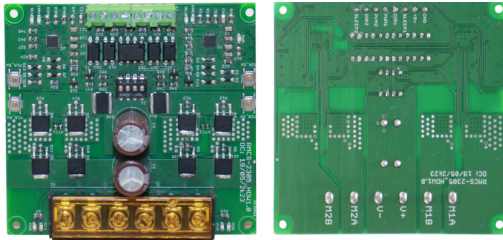


Figure 7. Rhino MDD20Amp Motor Driver

3.3.2 MD13S

MD13S Single Motor Driver Specifications

- Motors Controlled: 1 Brushed DC (Bi-directional).
- Operating Voltage: 6V to 30V.
- Current Limit: 13A continuous | 30A peak (10 seconds).
- Logic Inputs: 3.3V and 5V (GROVE compatible).
- PWM Frequency: Up to 20 kHz.



Figure 8. MD13S Motor Driver

3.4 IMU

An IMU (Inertial Measurement Unit) is an electronic device used to measure the motion and orientation of an object. Its main purpose is to determine how the object is moving and how its orientation is changing. The operating voltage of the IMU sensor used in this project is 3.3V.

Components:

Accelerometer: An accelerometer measures linear acceleration along the axes.

- X-axis → measures forward/backward acceleration
- Y-axis → measures left/right acceleration
- Z-axis → measures up/down acceleration

Gyroscope: A gyroscope measures the angular velocity of an object about each axis, determining how quickly the object is rotating or changing orientation. It measures angular velocity along three axes:

- The X-axis is also called Roll, and rotates about the left/right direction.
- The Y-axis is also called Pitch, and rotates about the up/down direction.
- The Z-axis is also called Yaw, and rotates about the left/right direction.

Magnetometer: Not all IMUs include a magnetometer. When present, it measures the surrounding magnetic field along three axes.

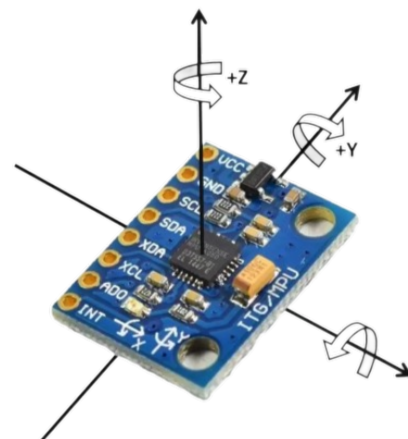


Figure 9. MPU6050

A magnetometer measures the strength and direction of the Earth's magnetic field. In an IMU, its main purpose is to provide a reference for direction, especially for yaw

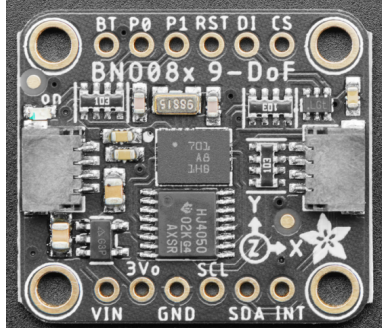


Figure 10. BNO085

Examples of IMU sensors:

- MPU-6050 (6-axis)
- BNO-085 (9-axis)

Note: If there is no magnetometer, the accelerometer and gyroscope together form a 6-axis IMU (i.e. 6 degrees of freedom); otherwise, it is a 9-axis IMU (i.e. 9 degrees of freedom).

BNO-085 Working Principle

- It is a 9-axis IMU sensor with a built-in magnetometer.
- Its major advantage is integrated sensor fusion, which helps determine the orientation of the bot.
- For accurate orientation estimation, the BNO085 is better suited to precise movement of the bot.

MPU-6050 Working Principle

- The MPU6050 is a 6-axis MEMS (Micro-Electro-Mechanical Systems) IMU that combines a 3-axis accelerometer and a 3-axis gyroscope on a single integrated circuit.
- The MPU6050 is significantly cheaper than the BNO085, often by a wide margin, making it the ideal choice for budget-constrained projects or when building multiple robots simultaneously.

3.5 Battery

LiPo (Lithium Polymer Battery)

A lightweight, rechargeable battery that uses a semisolid polymer electrolyte instead of a liquid one. It offers a higher specific energy (power-to-weight ratio) than standard lithium-ion batteries.

Key Parameters:

- **Cell Voltage:** A single cell (1S) has a nominal voltage of 3.7V. The total voltage determines the speed and power of the device (e.g., a 3S battery = 11.1V).
- **Cell Configuration (S/P):** Indicates how the cells are wired. S (Series) increases voltage, while P (Parallel) increases capacity (example: a 4S2P pack has 4 cells in series and 2 sets in parallel).
- **Capacity (mAh):** Measures the total stored energy, which determines runtime (example: a 1300mAh battery can safely output 1300mA continuously for one hour).
- **Discharge Rating (C-Rating):** Indicates how fast power can be safely drawn from the battery (example: a 95C rating means you can draw 95 times the battery's capacity). Higher C-ratings provide more burst power but make the battery heavier and more expensive.



Figure 11. Battery

3.6 LM2596 Buck Converter

The LM2596 is a switching-type DC-DC step-down module that efficiently steps down a higher DC input voltage into a stable, lower regulated output (such as 5V). It uses pulse-width modulation (PWM) switching at 150 kHz, filtered through an inductor and capacitor, to achieve typical efficiencies of 92–96%, far outperforming standard linear regulators in terms of heat dissipation. Additionally, it features an onboard multi-turn potentiometer for precise output voltage tuning and can deliver up to 3A of continuous load current.

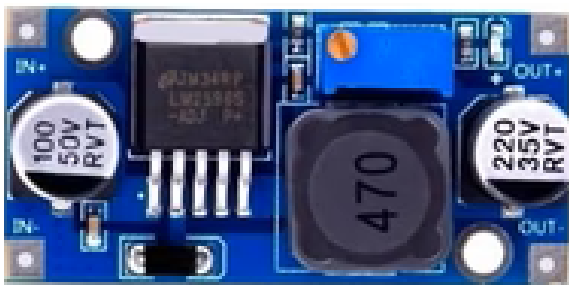
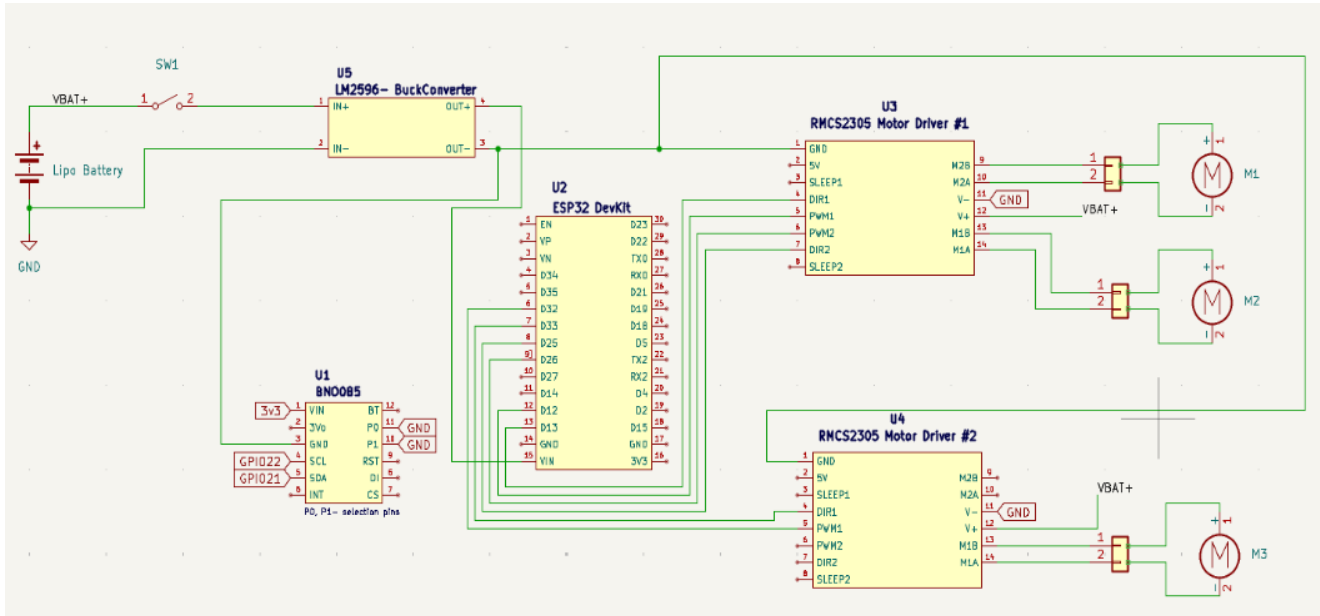


Figure 12. LM2596 Buck Converter

Key Parameters:

- **Input Voltage Range:** Accepts roughly 4.5V to 40V. For reliable operation, the input must be kept a few volts above the output (e.g., ideally 7V or higher for a 5V output) to maintain proper regulation.
- **Output Current:** Delivers up to 3A of continuous current. Sustained operation near this 3A limit, particularly when there is a large difference between input and output voltage, may require a small heatsink or airflow to manage heat.
- **Output Adjustment:** Available in fixed or adjustable variants. Adjustable models feature an onboard trimmer potentiometer to dial in the exact output voltage, which should be verified with a multimeter before connecting a load.
- **Applications:** Ideal for powering lower-voltage logic circuits (such as Arduino, Raspberry Pi, or various sensors) from higher-voltage sources like 12V or 24V batteries and adapters.

4. Circuit Diagram and Control Circuit



5. Web GUI and WebSocket

5.1 Web GUI

The Web GUI serves as a convenient alternative to the PS4 controller, allowing users to wirelessly operate the three-wheeled omnidirectional robot directly from a laptop or mobile phone. Designed for intuitive use, the interface mimics a standard controller and provides movement commands for forward, backward, lateral, and diagonal travel. Taking advantage of the robot's omnidirectional design, the GUI translates these inputs into immediate movement along the desired path without requiring the robot to rotate first.

The Web GUI is built using HTML, CSS, and JavaScript. HTML provides the basic structure of the controller, CSS arranges the buttons and makes the interface easier to use, and JavaScript handles the actual interaction with the robot. When a button is pressed or a movement command is given through the webpage, JavaScript sends that command to the ESP32 over a wireless connection.

5.2 WebSocket

An active WebSocket connection between the Web GUI and the ESP32 provides real-time, low-latency communication. For operational flexibility, the system accepts commands from both the Web GUI and a PS4 controller.

Control Workflow:

Web GUI → WebSocket → ESP32 → Inverse Kinematics + PID → Motor Drivers → 3 Motors → Movement

Motion Processing: The ESP32 extracts the desired V_x , V_y , and rotational velocity from the user's input.

Execution: Inverse kinematics determines the required speed for each wheel, which the motor drivers then apply to the motors.

GUI Capabilities: The interface features variable speed limits and an emergency stop, and it can be expanded for real-time monitoring of IMU heading, motor speeds, and connection status.

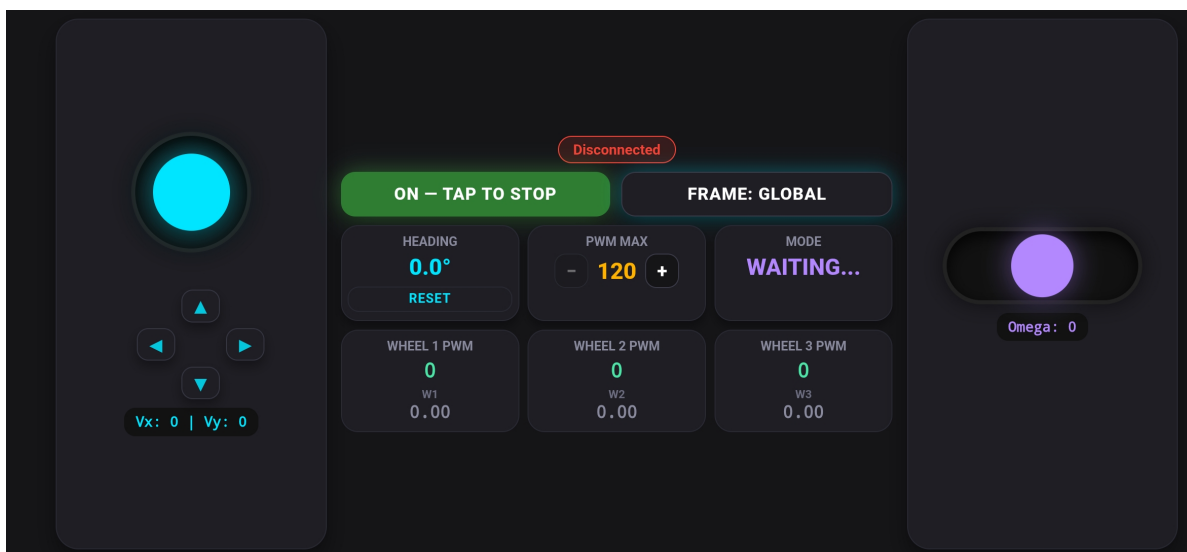


Figure 14. GUI interface

6. Communication Protocols

6.1 What are Communication Protocols?

Communication protocols define the rules and standards that allow different robotic components to exchange data accurately and reliably. In a robot, many parts must work together, such as sensors, microcontrollers, motor drivers, cameras, and computers. Without communication protocols, these components would have no way of knowing how to send, receive, or interpret data. A protocol specifies how data is formatted, how fast it is transmitted, how errors are detected, and how devices coordinate with each other. In simple terms, communication protocols act as a common language that lets robotic components exchange information.

6.2 Serial Peripheral Interface (SPI)

SPI (Serial Peripheral Interface) is a synchronous serial communication protocol used mainly for short-distance, high-speed communication between a microcontroller (master) and one or more peripheral devices (slaves). It is widely used in embedded systems because it provides full-duplex communication, meaning data can be transmitted and received simultaneously.

SPI generally uses a 4-wire interface:

- MOSI (Master Out Slave In) - transfers data from master to slave.
- MISO (Master In Slave Out) - transfers data from slave to master.
- SCK/SCLK (Serial Clock) - clock signal generated by the master to synchronize data transfer.

- CS/SS (Chip Select/Slave Select) - used by the master to select a particular slave. It is normally active LOW.

Master and Slave:

The master device controls SPI communication. It generates the clock, controls the data transfer, and selects which slave device should communicate. The slave devices are peripherals connected to the SPI bus. In a multi-slave system, MOSI, MISO, and SCK can be shared by all slaves, while each slave normally has its own CS/SS line. The master pulls the desired slave's CS line LOW to select it.

Advantages of SPI:

- High-speed data transfer
- Full-duplex communication
- Simple hardware interface
- Low protocol overhead
- Supports multiple slave devices
- Flexible clock configuration using CPOL and CPHA
- Widely supported by microcontrollers and peripheral ICs

6.3 I²C

I²C (Inter-Integrated Circuit) is a serial communication protocol that uses two lines: SDA (Serial Data) for transferring data and SCL (Serial Clock) for synchronizing communication.

Communication begins when the controller generates an I²C START condition. To create START, the controller first pulls SDA from HIGH to LOW while SCL is HIGH, and then pulls SCL LOW. This indicates that the controller has claimed the I²C bus.

When communication is finished, the controller generates an I²C STOP condition. It first re-

leases SCL so that it goes HIGH, then releases SDA, allowing SDA to transition from LOW to HIGH while SCL is HIGH. This releases the bus for the next communication.

6.4 UART

UART (Universal Asynchronous Receiver Transmitter) is a serial, asynchronous communication protocol used to exchange data between two devices. It is called asynchronous because there is no separate clock signal shared between the transmitter and receiver; instead, both devices must be configured with the same communication parameters, especially the baud rate.

UART generally uses two communication lines:

1. TX (Transmit): sends data from a device.
2. RX (Receive): receives data into a device.

6.4.1 UART Configuration

Before communication begins, both devices must be configured with compatible parameters. The most important of these is the baud rate, which specifies the number of symbols transmitted per second; a common configuration is 9600 baud.

The devices must also agree on the data length, parity, and number of stop bits. For example, if the transmitter sends 8-bit data, the receiver must also be configured for 8-bit data.

Advantages of UART:

- Requires only two communication lines for full-duplex communication.
- Simple hardware and software implementation; baud rate and data size can be configured.
- Supports full-duplex communication.
- Provides basic error detection using parity.
- Commonly used for serial communication and debugging.

Disadvantages of UART:

- Generally slower than high-speed synchronous protocols such as SPI.
- Start, stop, and optional parity bits introduce protocol overhead.
- Since there is no shared clock, both devices must be configured with compatible baud rates and frame parameters.
- Usually intended for communication between a small number of directly connected devices.

7. PID

7.1 Proportional Controller

The proportional component generates a correction that is directly proportional to the current error. As the error decreases, the correction also becomes smaller.

Effects of Proportional Gain (K_p):

Increasing the proportional gain improves the speed of the system's response, since larger corrective actions are generated. However, excessively high proportional gain can result in overshoot and oscillations, making the system unstable. Selecting an appropriate value of K_p is therefore essential for achieving a balance between responsiveness and stability.

Steady-State Error:

Steady-state error refers to the small difference that remains between the desired and actual output after the system has settled. Although proportional control can significantly reduce error, it may not eliminate it completely. This limitation is addressed through the introduction of the integral component of the PID controller.

Real-Life Example: Cruise Control System

Suppose a car's cruise control is set to maintain a speed of 60 km/h, but after some time the car stabilizes at 58 km/h instead.

Steady-State Error = $60 - 58 = 2$ km/h

Although the system has stabilized, it is still not reaching the exact desired speed. This remaining difference is the steady-state error.

Why Does Steady-State Error Occur?

Several factors can prevent a system from reaching the exact desired value:

- **Insufficient Controller Gain:** If the controller does not generate a strong enough corrective action, the system may

stop responding before the error reaches zero.

- **Proportional Control Limitation:** A controller using only proportional action generally reduces error but may not eliminate it completely.

This is one of the main reasons integral control is introduced in controllers.

Low K_p	High K_p
Slow response	Fast response
Large steady-state error	Small steady-state error
Stable (less overshoot)	Oscillatory (more overshoot)

Table 2. Characteristics of Proportional Gain (K_p)

7.2 Integral Controller (I)

The Integral Controller is the second component of the PID controller. Its primary purpose is to eliminate the steady-state error that remains after proportional control has acted.

Unlike the proportional controller, which considers only the current error, the integral controller continuously accumulates error over time. If a small error persists for a long duration, the integral action gradually increases the corrective output until the error is removed. As a result, the integral controller improves long-term accuracy and ensures that the system eventually reaches the desired setpoint.

What is Integral Gain?

Integral Gain (K_i) determines how strongly the controller reacts to accumulated error. It controls the speed at which the integral action builds up over time.

In a standard continuous PID control loop, K_i scales the integral of the system error, as shown in the combined PID equation below.

Low K_i	High K_i
Slow response	Fast response
Stable	Overshoot possible
Longer settling time	Faster error removal

Table 3. Characteristics of Integral Gain (K_i)

7.3 Derivative Controller

The Derivative Controller is the third component of the PID controller. Unlike the Proportional Controller, which reacts to the current error, and the Integral Controller, which accumulates past errors, the Derivative Controller responds to the rate at which the error is changing.

In simple terms, the derivative controller predicts future system behavior by observing how quickly the error is increasing or decreasing. Because of this predictive nature, derivative control improves system stability and reduces oscillations by reacting to the rate of change of error rather than waiting for a large error to appear. It anticipates what is likely to happen next and takes preventive corrective action.

Note: Without derivative action, a system may continue oscillating around the desired value before eventually settling. Derivative control slows the system as it approaches the target, reducing excessive movement and improving stability. As a result, the output reaches the desired value more smoothly and efficiently.

Derivative Gain (K_d):

Derivative Gain (K_d) determines how strongly the controller reacts to the rate of change of error. It controls the amount of damping introduced into the system.

A properly selected value of K_d helps the system approach the desired value smoothly without excessive overshoot. K_d acts as a braking mechanism: just as brakes prevent a vehicle from overshooting a turn, derivative control prevents the system from overshooting the desired value.

Low K_d	High K_d
Faster	Slower
More Overshoot	Less Overshoot
Oscillatory	Stable
Less Damping	More Damping

Table 4. Characteristics of Derivative Gain (K_d)

7.4 Combining P, I and D

Each controller addresses a different part of the problem:

- **P:** Responds to the current error and provides immediate corrective action.
- **I:** Eliminates steady-state error and improves long-term accuracy.
- **D:** Predicts future behavior and reduces oscillations.

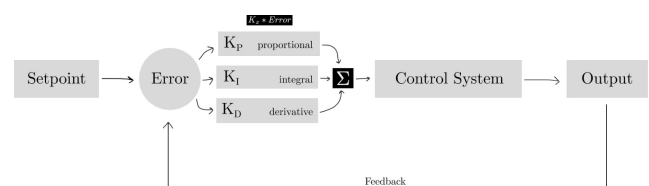


Figure 15. Block Diagram of PID

A PID controller combines the strengths of all three control actions into a single system. The proportional term provides an immediate response to the current error, the integral term ensures that small persistent errors are eventually removed, and the derivative term prevents excessive correction by predicting future system behavior.

As a result, the system becomes faster, more accurate, and more stable than when using any

single controller alone. This is why PID controllers are widely used in applications where precise control and reliable performance are required.

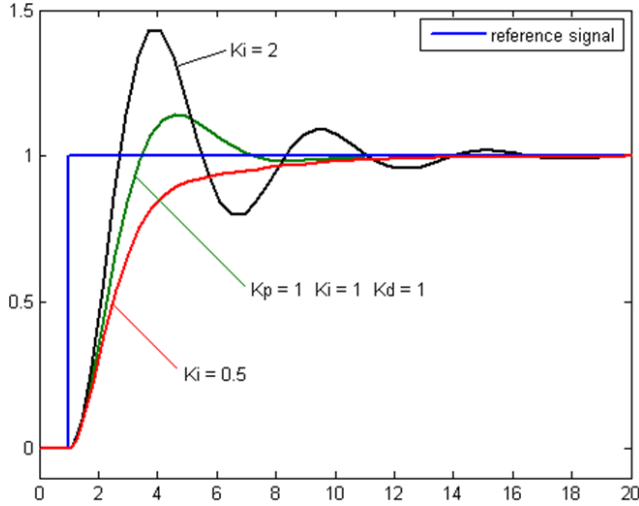


Figure 16. Graph of PID

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

- $e(t)$: Current error (Set Point – Process Variable)
- $\int_0^t e(\tau) d\tau$: Accumulated historical error over time

- K_i : Integral gain scaling factor

Finding the perfect balance between these three components requires careful tuning of the K_p , K_i , and K_d parameters. For our omnidirectional robot, precise tuning ensures that the chassis maintains its exact heading without drifting or violently oscillating during complex holonomic movements.

Key Functions & Tuning

- **Eliminates Steady-State Error:** While proportional control (K_p) often leaves a constant residual offset, the integral term continues to accumulate this remaining error until the output drives the steady-state error to zero.
- **High K_i (Over-tuning):** Results in faster error elimination, but increases overshoot, causes system oscillations, and risks integral windup or instability.
- **Low K_i (Under-tuning):** Ensures greater stability, but results in sluggish response times and prolonged steady-state offset.

8. References

- Inverse Kinematics - "A General Inverse Kinematic Formulation and Control Schemes for Omnidirectional Robots" by Indrazno Siradjuddin, Gilang Al Azhar, Sapto Wibowo, Ferdian Ronilaya, Cahya Rahmad, and Erfan Rohadi
- ESP32 - Datasheet of ESP32 by Espressif
- Motor - Datasheet of DC Planetary Geared Motor by Robokits
- RMCS2305 - Datasheet of RMCS2305 by Rhino Motors
- MD13S - Datasheet of MD13S by Cytron Technologies
- IMU - Article on IMU: An Introduction, by Advanced Navigation
- Arduino Code Reference - Arduino IDE library documentation
- BNO085 - Datasheet of Adafruit 9-DOF Orientation IMU Fusion Breakout - BNO085, created by Bryan Siepert
- HTML, CSS, JavaScript - YouTube videos by Thapa Technical (HTML), CodeHelp (CSS), and WsCube Tech (JavaScript)
- WebSocket - Arduino IDE libraries
- Communication Protocols - Articles from GeeksforGeeks
- PID - MATLAB Youtube Series